

Repository-Grounded Technical Report for **main_3** Corrosion Quantification, Hidden-Damage Proxy Modeling, and Threshold-Based Proxy RUL

March 17, 2026

Contents

1	Problem Framing	3
2	Repository-Grounded Pipeline Overview	4
2.1	Implementation Consequences That Matter Scientifically	8
3	Data Description and Data Dictionary	8
3.1	Target Availability and Label Balance	9
3.2	Important Canonical Fields	9
3.3	Naming, Treatment, and Unit Ambiguities	12
4	Feature Engineering	13
4.1	Implemented Image Preprocessing	13
4.2	Engineered Feature Families	13
4.3	Feature-Target Agreement and Tautology Risk	15
4.4	Feature Inclusion and Exclusion Rules	16
5	Technical Challenges and Implemented Responses	17
6	Modeling Methodology	18
6.1	Corrosion Stage	18
6.2	Damage Stage	19
6.3	Degradation Fitting	20
6.4	Proxy-RUL Estimation	20
7	Split Strategies and Leakage Control	21
7.1	Why Row-Wise Random Splitting Would Be Invalid	21

7.2	Implemented Split Regimes	21
7.3	Continuous-Target Performance Under Leakage-Aware Splits	21
7.4	Categorical Corrosion Performance	22
8	Results and Interpretation	22
8.1	Strongly Supported Results	22
8.2	Weak, Negative, and Mixed Results	23
8.3	Degradation and Proxy-RUL Outputs	25
9	What the Model Can and Cannot Learn	26
9.1	What Is Directly Visible	26
9.2	What Is Only Indirectly Inferable	26
9.3	What Is Hidden and Not Reliably Recoverable Here	26
9.4	Why Some Targets Are Easier Than Others	27
9.5	Why Metadata Can Outperform Image Features	27
9.6	Scientific Implication	27
10	Traceability to the Original Thesis and Documentation	27
11	Limitations	29
12	Future Work	30
A	Meeting-Defense Questions and Concise Evidence-Based Answers	30

Abstract

This report documents what the repository state of `main_3` actually implements. The evidence base is the executable code in `src/` and `scripts/`, the configuration in `configs/default.toml`, the saved tables and figures in `outputs/` and `reports/`, the pipeline logs in `logs/`, and the thesis and conference PDFs stored in `../Documentation`. The implemented system reconstructs a canonical specimen-time dataset, extracts classical image descriptors from 792 PNG images, trains image-only models for visible corrosion targets, trains mixed-input regressors for structural proxy targets on only 48 late-stage rows, fits specimen-wise degradation curves, and estimates threshold-based proxy remaining useful life (RUL). The repository does not support direct supervised image-to-RUL learning, and the downstream hidden-damage/RUL stages are not end-to-end image-only because the saved damage models use manual corrosion labels and metadata as inputs. All conclusions below are therefore limited to what this implemented pipeline can defensibly support.

1 Problem Framing

The implemented system solves a staged condition-assessment problem rather than a single end-to-end prognostics problem. At repository level, the workflow is:

1. reconstruct a canonical record of specimen identity, campaign, treatment, image path, visible corrosion labels, and sparse structural labels;
2. extract interpretable image features from each photograph;
3. predict visible corrosion targets from image features;
4. estimate hidden damage proxies and residual load capacity from a mixture of metadata, manual corrosion labels, and selected image features;
5. fit specimen-wise degradation curves; and
6. define proxy RUL as the first forecasted crossing of selected engineering thresholds.

Direct supervision is asymmetric across stages. Visible corrosion supervision is dense: the canonical dataset contains 792 image rows with numeric surface corrosion labels, numeric peak corrosion labels, ordinal corrosion categories, and peak-rust location. Structural supervision is sparse: only 48 rows, exactly one per specimen, contain wire-area-loss and ultimate-load labels, and those structural rows occur only at weeks 28 and 36. True failure times are absent. Therefore:

- direct image-to-corrosion learning is implemented and supported for current visible corrosion targets;
- direct image-to-hidden-damage learning is not cleanly implemented as an image-only task, because the saved damage models consume manual corrosion labels and metadata in addition to selected image features;
- direct image-to-RUL learning is not supported by either the code or the dataset because no time-to-failure target exists.

The exact scope of `main_3` is therefore narrower than a fully autonomous structural prognosis system. Scientifically, the repository supports image-based interpolation of current visible corrosion on new specimens under related acquisition conditions. It supports only exploratory inference of hidden damage and threshold-status trajectories, because those later stages are driven by sparse structural supervision, campaign-linked metadata, and model-generated pseudo-trajectories rather than repeated structural observations over life.

2 Repository-Grounded Pipeline Overview

Table 1 reconstructs the implemented pipeline stage by stage from the code. The most important practical point is that the repository contains several stages that are diagnostic or bookkeeping layers rather than strict data dependencies for later steps.

Table 1: Implemented pipeline stages reconstructed from `scripts/` and `src/`.

Stage	Main code	Input	Output	Purpose, key design choices, and main limitations
Inspection	<code>scripts/01_inspect_dataset.py</code> , <code>src/data/canonical.py</code>	<code>Data/Images_Dataset</code> files; PNG directory <code>../Data/Images_dataset</code>	In-memory <code>dataset_index</code> ; <code>reports/dataset_inspection</code>	Loads the spreadsheet, treats inspection spreadsheet row as semantic headers, parses record IDs, scans the image directory, merges rows to image files, and validates duplicates or missing joins. Limitation: validation is structural and syntactic only; it cannot resolve semantic ambiguities in label definitions.
Canonicalization	<code>scripts/02_build_inspection.py</code> , <code>src/data/io.py</code>	Inspection output	<code>outputs/canonical</code> <code>.parquet</code> , <code>dataset_summary.csv</code>	Raw dataset raw, columns to canonical names, derives campaign and sequence fields, rescales wire loss when the raw maximum is ≤ 1.5 , and stores row-level flags such as <code>has_structural_labels</code> . Limitation: campaign is inferred from metadata and is therefore both informative and potentially confounding downstream.

Continued on next page

Stage	Main code	Input	Output	Purpose, key design choices, and main limitations
Preview-only preprocessing	<code>scripts/03_preprocess_images.py</code> <code>src/visualization/plotting.py</code>	Cassini imageset, <code>n_plots</code> and original image paths	24 preview PNG panels and <code>outputs/preprocess</code>	Generates diagnostic three-panel previews (ROI, previsualized ROI, rust-mask overlay). This stage does <i>not</i> create persistent preprocessed inputs for later stages; later feature extraction reruns preprocessing internally on the original images.
Feature extraction	<code>scripts/04_extract_features.py</code> <code>src/features/</code>	Cassini dataset and original PNGs	<code>outputs/image_features.csv</code> , <code>features.parquet</code>	Applies Python preprocessing pipeline, computes rust likelihood, adaptive thresholding, morphology cleanup, 12 longitudinal segment features, color statistics, histograms, component geometry, and texture descriptors. One failure (E01-20240508-17W) is retained with <code>feature_extraction_failed=1</code> and median-filled numeric features.

Continued on next page

Stage	Main code	Input	Output	Purpose, key design choices, and main limitations
Corrosion modeling	<code>scripts/05_train_corrosion.py</code> <code>src/models/corrosion.py</code>	Corrosion dataset, <code>corrosion.py</code> with all image features	Metrics, fold predictions, full-data saved estimators, and <code>corrosion_state_estimator.py</code>	Trains image-only regressors and classifiers for four visible corrosion states under three split strategies. All image features are used; no explicit feature pruning is applied. Final saved models are chosen only by <code>group_shuffle</code> performance.
Damage modeling	<code>scripts/06_train_damage.py</code> <code>src/models/damage.py</code>	Damage dataset, <code>damage.py</code> of the canonical dataset, selected metadata, manual corrosion labels, and selected image features	Metrics, fold predictions, full-data saved estimators, and <code>damage_state_estimator.py</code>	Estimates <code>wire_area_loss_pct</code> and <code>ultimate_load_kn</code> . Important limitation: the damage feature list excludes the upstream <code>pred_*</code> corrosion outputs, so the saved damage models are not driven by predicted corrosion states; they use manual corrosion labels available in the canonical dataset.

Continued on next page

Stage	Main code	Input	Output	Purpose, key design choices, and main limitations
Degradation fitting	scripts/07_fit_degradation_models/ src/degradation/	07_data_loader.py CMA-ES model plus full damage-state estimates	degradation_curves.parquet degradation_forecasts.parquet	Fit a curve per specimen per target by choosing the lowest in-sample RMSE among linear, exponential, logarithmic, and piecewise-linear families. Surface corrosion curves are fitted to observed labels; hidden-damage and load curves are fitted to model-generated estimates at all weeks. There is no monotonicity constraint.
Proxy RUL	scripts/08_estimating_rul/ src/rul/	08_estimating_rul.py Interpolated state plus degradation forecasts	rul_estimates.csv rul_estimates.parquet, rul_trajectories.parquet	Defines proxy RUL as first forecasted crossing of selected wire-loss, load, or health thresholds. This is explicitly threshold-status analysis, not supervised failure-time prediction. Output failure_probability is a weighted proxy score, not a calibrated probability.

Continued on next page

Stage	Main code	Input	Output	Purpose, key design choices, and main limitations
Report generation	<code>scripts/09_generate_reports.py</code>	Saved reports, predictions, forecasts, and RUL tables	Markdown summary plus PNG figures in <code>reports/figures/</code>	Generates parity plots and bar charts, but only the displayed <code>group_shuffle</code> figures are used in the saved summary report. Stricter split results exist in CSV files and must be read directly to avoid overclaiming robustness.

2.1 Implementation Consequences That Matter Scientifically

Three repository facts materially change how the pipeline should be described.

First, the preprocessing script is not a persistent data-transformation stage. It is a diagnostic visualization stage. The same preprocessing logic is re-executed later inside `extract_image_features()`.

Second, the corrosion stage is not functionally upstream of the damage stage in the saved implementation. The damage design table merges `corrosion_state_estimates.parquet`, but the selected feature list in `_damage_feature_columns()` does not use the `pred_*` columns. The damage models therefore rely on manual corrosion labels, not on predicted corrosion states.

Third, the degradation and proxy-RUL stages for hidden damage are model-on-model stages. They fit curves to `estimated_wire_area_loss_pct` and `estimated_ultimate_load_kn`, which are themselves outputs of full-data damage regressors. Those curves are not repeated structural measurements collected at multiple weeks.

3 Data Description and Data Dictionary

The canonical dataset contains 792 rows, 48 specimens, 7 treatment codes, and 2 campaigns, with 24 specimens in each campaign. The two inferred campaign identifiers are `campaign_2022_mesh7_nacl0.035_age201` and `campaign_2024_mesh4_nacl0.050_age260`. Campaign A spans weeks 0 to 28 with 360 rows; Campaign B spans weeks 0 to 36 with 432 rows. Structural labels exist in only 48 rows, exactly one row per specimen, at weeks 28 or 36.

Quantity	Value
Image rows	792
Specimens	48
Campaigns	2
Treatment codes	7
Week range	0–36
Structural rows	48
Structural coverage	6.06%
Structural weeks	28 and 36
Duplicate <code>record_id</code> values	0
Duplicate specimen/week combinations	0
Invalid parsed IDs	0
Missing images after spreadsheet-image merge	0
Feature extraction failures logged	1

Table 2: Dataset summary reconstructed from `outputs/dataset_summary.csv`, `outputs/canonical_dataset.parquet`, and `logs/04_extract_features.log`.

3.1 Target Availability and Label Balance

Target	Count	Mean	Median	Min	Max
<code>surface_total_rust_pct</code>	792	2.850	0.347	0.000	53.412
<code>peak_rust_pct</code>	792	9.592	2.239	0.000	84.304
<code>peak_rust_location_cm</code>	792	9.535	6.500	0.510	23.480
<code>wire_area_loss_pct</code>	48	22.233	21.045	0.000	56.980
<code>ultimate_load_kn</code>	48	2.180	2.250	1.600	2.870

Table 3: Observed target distributions in the canonical dataset.

Category	1	2	3	4	5
<code>surface_total_rust_category</code>	493	51	38	77	133
<code>peak_rust_category</code>	304	159	72	122	135

Table 4: Ordinal corrosion-category imbalance. The repository provides category values but does not document numeric cutoffs for the 1–5 scale.

The category balance in Table 4 matters for interpretation. Surface-category accuracy can look strong even when macro-F1 remains only moderate, because category 1 dominates 493 of the 792 rows. The code treats the category columns as supervised labels, but the repository does not contain the rule that maps those 1–5 categories to physical percentage thresholds.

3.2 Important Canonical Fields

Table 5: Core identifiers, metadata, and housekeeping fields in the canonical dataset.

Column	Meaning	Unit / type	Origin	Role and notes
<code>record_id</code>	Image/row identifier following the parsed pattern <code>specimen-date-weekW</code>	string	Spreadsheet → canonical rename	Primary row key. The code validates the format with a regular expression.
<code>specimen_id</code>	Specimen identity extracted from <code>record_id</code>	string	Derived from <code>record_id</code>	Group key for leakage-aware splitting and time ordering.
<code>capture_date</code>	Image acquisition date embedded in <code>record_id</code>	datetime	Derived from <code>record_id</code>	Used for campaign inference and chronological ordering.
<code>week</code>	Exposure week embedded in <code>record_id</code>	integer weeks	Derived from <code>record_id</code>	Temporal coordinate for splits, degradation fitting, and RUL.
<code>series_family</code>	Alphabetic family prefix of the specimen ID	string	Derived from <code>specimen_id</code>	High-level grouping such as S, D, E, F, or G.
<code>series_label</code>	Shorter specimen-series label	string	Derived from <code>specimen_id</code>	Used as categorical metadata in the damage stage.
<code>treatment_code</code>	Treatment code from the spreadsheet	string	Spreadsheet	Authoritative treatment label for split logic. Important: filename substrings are not reliable substitutes. For example, <code>S1MI01</code> has <code>treatment_code=NO</code> .
<code>treatment_name</code>	Human-readable treatment label	string	Derived via mapping table	Convenience label only; no modeling stage uses it directly.
<code>campaign_id</code>	Campaign identifier composed from year, mesh count, NaCl%, and ageing days	string	Derived	Used directly in evaluation and damage modeling. Scientifically, it compresses multiple acquisition and experimental settings into one shortcut variable.
<code>n_steel_mesh</code>	Number of steel meshes	integer	Spreadsheet	Metadata. On the structural subset this is perfectly confounded with campaign.

Continued on next page

Column	Meaning	Unit / type	Origin	Role and notes
ageing_days	Ageing duration associated with the campaign	integer days	Spreadsheet	Metadata. Also perfectly confounded with campaign on the structural subset.
nacl_pct	Salinity value	percent or fraction as stored	Spreadsheet	Metadata. Used in campaign inference and damage modeling.
cover_failure_surface_mm	Surface cover at the failure surface	mm	Spreadsheet	Metadata and possible structural proxy.
image_path	Absolute path to the PNG file	string	Image discovery + merge	Used by the preview and feature-extraction stages.
image_filename	PNG filename	string	Image discovery + merge	Traceability only.
image_join_status	Merge status after outer joining spreadsheet rows to image files	categorical	Derived	All rows are both in the saved outputs, so the spreadsheet-image join is complete.
image_exists	Whether an image path is present after the merge	boolean	Derived	All 792 rows are true in the saved outputs.
has_structural_labels	Whether wire loss or ultimate load is present	boolean	Derived	Structural subset selector; true in 48 rows only.
sequence_order	Within-specimen chronological index	integer	Derived	Computed after sorting by specimen, week, and record ID.
duplicate_record_ID	Duplicate record-ID flag	boolean	Derived	All false in the saved outputs.
duplicate_specimen_ID	Duplicate specimen/week flag	boolean	Derived	All false in the saved outputs.

Table 6: Targets and derived target-side fields in the canonical dataset.

Column	Meaning	Unit / type	Origin	Role and notes
surface_total_rust_pct	Top-down visible rust percentage on the surface	percent	Spreadsheet	Dense supervised regression target. Also used directly as an input to the damage stage.

Continued on next page

Column	Meaning	Unit / type	Origin	Role and notes
<code>surface_total_rust_category</code>	Ordinal severity code for total visible rust	ordinal integer 1–5	Spreadsheet	Dense supervised classification target. Repository does not define the exact class boundaries.
<code>peak_rust_pct</code>	Peak localized rust percentage	percent	Spreadsheet	Dense supervised regression target and an input to the damage stage.
<code>peak_rust_category</code>	Ordinal severity code for peak rust	ordinal integer 1–5	Spreadsheet	Dense supervised classification target. Again, class definitions are not recoverable from the repository.
<code>peak_rust_location_cm</code>	Location of the peak-rust zone along specimen length	cm	Spreadsheet	Dense manual label used as metadata in the damage stage. No dedicated predictive model is trained for this target in <code>main_3</code> .
<code>wire_area_loss_raw</code>	Raw structural area-loss field from the spreadsheet	numeric	Spreadsheet	Sparse structural measurement. Values top out at 0.5698, which motivated the scaling rule described below.
<code>wire_area_loss_pct</code>	Rescaled wire-loss value used by later stages	percent	Derived	Computed by multiplying <code>wire_area_loss_raw</code> by 100 when the dataset maximum is ≤ 1.5 . This scaling is implemented in code, not separately documented in the spreadsheet.
<code>ultimate_load_kn</code>	Measured ultimate load	kN	Spreadsheet	Sparse structural regression target, available only in the 48 structural rows.

3.3 Naming, Treatment, and Unit Ambiguities

Two ambiguities are important enough to state explicitly in the report. First, specimen strings cannot be treated as authoritative treatment labels. The spreadsheet `treatment_code` must remain the source of truth, because several IDs contain treatment-like substrings that do not

match the actual treatment assignment. Second, the raw wire-loss field is unit-ambiguous in isolation. The saved code resolves this by treating values such as 0.5698 as fractions and converting them to 56.98%, because the downstream thresholds are defined at 20%, 25%, and 30%.

4 Feature Engineering

4.1 Implemented Image Preprocessing

The current image pipeline is implemented in Python, not in GIMP or MATLAB. Each image is loaded as RGBA, composited onto a white background, converted to RGB, cropped by fixed fractions (3% left, 3% right, 8% top, 8% bottom), illumination-normalized, converted to a rust-likelihood map, and then segmented into a rust mask.

The normalization step rescales each RGB channel between its 2nd and 98th percentiles, converts to HSV, and then equalizes the value channel with adaptive histogram equalization. The rust-likelihood score is a weighted combination of red dominance, warm dominance, Lab chromatic coordinates, HSV saturation, and two hue-centered Gaussian terms:

$$\text{rust_score} = 0.24 (R-G)_+ + 0.18 (R-B)_+ + 0.18 a_{\text{norm}} + 0.14 b_{\text{norm}} + 0.14 S + 0.07 g(h; 0.03) + 0.05 g(h; 0.08),$$

with the score suppressed when the normalized value channel is nearly white ($V \geq 0.99$). Segmentation then uses

$$\text{threshold} = \text{clip}(\max(\text{Otsu}(\text{rust_score}), \mu + 0.35\sigma), 0.22, 0.82),$$

followed by removal of small objects and holes, binary opening with a disk of radius 1, and binary closing with a disk of radius 2.

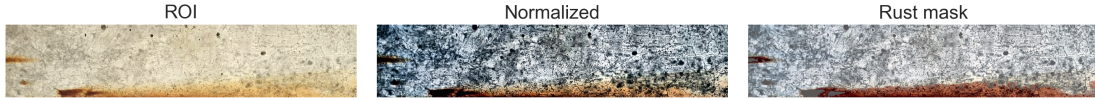


Figure 1: Example of the `main_3` preprocessing output written by the preview stage: cropped ROI, illumination-normalized ROI, and rust-mask overlay. This figure documents the current Python implementation, not the thesis GIMP/BIMP workflow.

4.2 Engineered Feature Families

The saved feature table contains 792 rows and 92 columns including `record_id`, so there are 91 engineered numeric features. Table 7 groups the important families by intended physical meaning and failure mode.

Table 7: Main image-derived feature families implemented in `src/features/extractor.py`.

Family	Variables	How computed	Why it may correlate	Main weaknesses
Mask-level corrosion extent	<code>rust_area_pct_feature</code> , <code>peak_rust_pct_feature</code>	Fraction of rust-mask pixels over the full ROI or the most active longitudinal segment	Larger and denser rust coverage should track more visible corrosion	Highly sensitive to segmentation mismatch, lighting, and threshold choice; can become tautological if the manual label was produced with a very similar mask, but the observed correlations here are only weak to modest.
Longitudinal localization	<code>peak_rust_location_pct_feature</code> , <code>rust_segment_00_pct</code> , <code>rust_segment_11_pct</code>	Relative, into 12 equal-width, non-overlapping segments; the segment with maximum mask ratio is mapped to a 24 cm specimen length	Encodes where corrosion concentrates along the specimen	Not equivalent to the thesis strip method. With 12 segments over 24 cm, the current spatial granularity is about 2 cm, not 1 cm with overlap.
Distribution statistics	<code>rust_distribution_std</code> , <code>range</code> , <code>entropy</code>	Standard deviation, max-min span, and entropy of segment rust ratios	Distinguishes diffuse staining from sharply localized attack	Campaign- and preprocessing-dependent; not directly interpretable as physical damage depth.
Component morphology	<code>rust_component_count</code> , <code>largest_area_pct</code> , <code>mean_area_pct</code> , <code>mean_eccentricity</code>	Connected components of the binary rust mask using <code>skimage.measure.regionprops</code>	Captures patch multiplicity, clustering, and elongated regions/streaks	Very sensitive to over- or under-segmentation; one failed image receives median-filled morphology values.
Rust-score statistics	<code>rust_score_mean</code> , <code>rust_score_std</code> , <code>rust_segmentation_threshold</code>	Summary statistics of the continuous thresholded likelihood map and the chosen threshold	Encodes whether the image globally appears more rust-like, and how separable rust is from the background	Threshold is partly acquisition-dependent and may encode lighting or campaign differences rather than corrosion alone.

Continued on next page

Family	Variables	How computed	Why it may correlate	Main weaknesses
Global color statistics	RGB, HSV, Lab means and standard deviations	Channel moments computed on the normalized ROI	Rust changes color balance, saturation, and chromaticity	Can capture illumination residue or campaign-specific acquisition style. In the saved surface-total-rust random forest, <code>rgb_b_mean</code> has the highest feature importance.
HSV histograms	<code>hist_h_bin_*</code> , <code>hist_s_bin_*</code> , <code>hist_v_bin_*</code>	8-bin normalized histograms on hue, saturation, and value	More flexible description of the color distribution than global means alone	Redundant with other color features and potentially unstable under preprocessing mismatch.
Texture	GLCM contrast, dissimilarity, homogeneity, energy, correlation; LBP bins	Gray image resized to 96×384 , GLCM at distances 1, 4, 8 and angles $0^\circ, 45^\circ, 90^\circ$; uniform LBP with $P = 8$, $R = 1$	Corrosion patches alter roughness, local contrast, and speckle patterns	Texture can respond to shadows, cracks, pores, and imaging artifacts, not only corrosion.
Acquisition and diagnostics	<code>image_height_px</code> , <code>image_width_px</code> , <code>feature_extraction_failed</code> , <code>edge_density</code> , grayscale mean/std	Geometry fields inherited from the <code>ROI</code> and other generic image statistics	May encode acquisition consistency or crack-like edge density	Geometry is not a material property and may act as a shortcut. <code>feature_extraction_failed</code> flags a single corrupted image retained in the dataset.

4.3 Feature-Target Agreement and Tautology Risk

Some feature names look like direct substitutes for spreadsheet targets, but the saved outputs show that the implemented features are not simple numerical copies of the manual labels. Table 8 summarizes the clearest diagnostics.

Manual label	Most obvious engineered proxy	Pearson r	Interpretation
surface_total_rust_pct	rust_area_pct_feature	0.257	The hard mask area is only weakly aligned with the manual total-rust label. The stronger univariate correlates are saturation-histogram bins, not the raw mask area.
peak_rust_pct	peak_rust_pct_feature	0.183	The current 12-segment peak proxy does not reproduce the manual peak label closely.
peak_rust_location_cm	peak_rust_location_cm_feature	0.248	The implemented location proxy is informative but coarse. It uses 12 non-overlapping segments over 24 cm.
surface_total_rust_pct	best single current feature: hist_s_bin_05	0.568	Color-distribution descriptors currently align better than the nominal area proxy.
peak_rust_pct	best single current feature: hsv_s_mean	0.571	Peak corrosion is currently tracked more by saturation structure than by the nominal peak-mask percentage.

Table 8: Agreement diagnostics computed from the saved canonical and image-feature tables. These values show some interpretability, but they also show that the current Python preprocessing is not reproducing the thesis strip-threshold labels exactly.

The repository therefore supports a nuanced statement about tautology. Visible corrosion modeling is partly reconstructive in the sense that the features were designed to respond to rust-colored and rust-textured regions. However, the weak alignment of the most obvious engineered proxies with the manual labels indicates that the current implementation is not merely reading off the target from an identical mask; it is learning from a broader, noisier image-description space.

4.4 Feature Inclusion and Exclusion Rules

The corrosion stage uses every numeric feature generated by the extractor, including generic image-geometry fields and the failure flag. There is no automatic redundancy pruning, no correlation filter, and no downweighting of potentially unstable features. The damage stage behaves differently: it uses a manually curated list of metadata fields, manual corrosion labels, and selected image features. Important exclusions are just as informative as inclusions:

- the merged `pred_surface_total_rust_pct`, `pred_peak_rust_pct`, and category prediction columns are *not* used by the saved damage models;
- no dedicated model is trained for `peak_rust_location_cm`, even though that field is later used as an input to the damage stage;
- the feature-extraction failure row is retained rather than dropped, and its missing numeric values are filled with per-column medians.

5 Technical Challenges and Implemented Responses

Table 9: Main implementation challenges visible in the repository and how they were handled.

Challenge	How detected	Implemented response	Residual risk
Spreadsheet header row is stored as data	<code>load_excel_table()</code> reads the first row as semantic headers	The loader promotes row 0 to the header line and numeric-casts the remaining columns	If the spreadsheet layout changes, parsing will silently break unless validation is extended.
File alignment and ID parsing	The code parses <code>record_id</code> with a strict regex and validates duplicates or invalid IDs	Outer merge between spreadsheet rows and discovered PNGs, with explicit flags for bad IDs or missing images	Structural alignment is clean in the current snapshot, but semantic mislabeling cannot be detected by syntax checks alone.
Specimen names do not equal treatment assignment	Canonical data show examples such as <code>S1MI01</code> → <code>N0</code> and <code>S4SAVF01</code> → <code>N0</code>	Treatment is taken from the spreadsheet, not inferred from the ID string	Any external analysis that infers treatment from filenames alone would be wrong for part of the dataset.
Wire-loss unit ambiguity	The raw structural field peaks at 0.5698 despite downstream thresholds being 20%, 25%, 30%	The code multiplies the raw values by 100 whenever the dataset maximum is ≤ 1.5	This is a reasonable repository-grounded assumption, but it remains an assumption rather than an independently documented unit conversion.
Corrupted image	<code>logs/04_extract_features.log</code> retained records <code>E01-20240508-17W</code> as unreadable	Structure was obtained with <code>feature_extraction</code> and median-filled numeric features	The sample stays in downstream modeling and failed by estimation, so one record carries synthetic feature values.
Repeated-measures leakage risk	Each specimen appears in 15–18 time points	Split logic groups by specimen for <code>group_shuffle</code> ; treatment and campaign holdouts are also evaluated	A naive row-wise split would be invalid. The repository avoids that specific error.
Structural supervision sparsity	Only 48 rows have structural labels, exactly one row per specimen	Damage models are restricted to those 48 rows and reported separately from corrosion models	There is no repeated structural trajectory to validate hidden-damage progression over time.

Continued on next page

Challenge	How detected	Implemented response	Residual risk
Campaign confounding in structural rows	On the 48 structural rows, week , ageing_days , nacl_pct , and n_steel_mesh are perfectly campaign-linked	The code evaluates leave-one-campaign-out and leave-one-treatment-out performance, exposing collapse under domain shift	The confounding remains in the fitted models; it is measured, not removed.
Preprocessing mismatch relative to thesis workflow	Thesis chapter 7 describes GIMP/BIMP preprocessing and MATLAB RGB masks; current code uses a Python adaptive pipeline	The repository implements a new, reproducible Python pipeline rather than reproducing the exact thesis steps	Differences in color normalization, cropping, and strip logic can plausibly explain part of the weak feature-label alignment.
Downstream label shortcut	Damage feature list includes manual corrosion labels and excludes upstream corrosion predictions	The repository still saves corrosion-state estimates, but they are not consumed in the damage models	The saved hidden-damage and proxy-RUL stages cannot be defended as end-to-end image-only inference.

6 Modeling Methodology

6.1 Corrosion Stage

The corrosion stage merges the canonical dataset with all 91 engineered image features and trains four supervised models:

- regression targets: `surface_total_rust_pct`, `peak_rust_pct`;
- classification targets: `surface_total_rust_category`, `peak_rust_category`.

All corrosion models are built inside a common scikit-learn pipeline with median imputation, z-score standardization for numeric columns, and one-hot encoding for categorical columns. In practice, the corrosion feature table is numeric, so the categorical branch is inert here.

Candidate regressors are:

- linear regression;
- random forest regressor with `n_estimators=400`, `min_samples_leaf=2`;
- histogram gradient boosting regressor with `learning_rate=0.05`, `max_depth=6`, `max_iter=400`;
- MLP regressor with hidden layers (128, 64), ReLU activation, Adam solver, early stopping, and `max_iter=600`.

Candidate classifiers are:

- logistic regression with `max_iter=2000`;
- random forest classifier with `n_estimators=400, min_samples_leaf=2`;
- histogram gradient boosting classifier with `learning_rate=0.05, max_depth=6, max_iter=300`;
- MLP classifier with hidden layers (128, 64), ReLU, early stopping, and `max_iter=600`.

Model selection is simple and important to state accurately. The repository evaluates all three split strategies, but `select_best_model()` picks the saved estimator using only mean `group_shuffle` performance: lowest MAE for regression and highest macro-F1 for classification. The final saved full-data models are therefore:

- `surface_total_rust_pct`: random forest;
- `peak_rust_pct`: MLP regressor;
- `surface_total_rust_category`: logistic regression;
- `peak_rust_category`: MLP classifier.

6.2 Damage Stage

The damage stage begins with the full design table produced by merging canonical metadata, all image features, and corrosion-state estimates. However, the saved feature subset is hard-coded and excludes the upstream predicted corrosion columns. The actual damage inputs are:

- metadata: `week, n_steel_mesh, ageing_days, nacl_pct, cover_failure_surface_mm, treatment_code, campaign_id, series_label`;
- manual corrosion labels: `surface_total_rust_pct, peak_rust_pct, surface_total_rust_category, peak_rust_category, peak_rust_location_cm`;
- selected image features: `rust_area_pct_feature, peak_rust_pct_feature, peak_rust_location_cm_feature, rust_distribution_std, rust_component_count, rust_component_largest_area_pct, rust_score_mean, rust_score_std, edge_density, texture_contrast_mean, texture_homogeneity_mean, texture_energy_mean, lab_a_mean, lab_b_mean, hsv_s_mean, and hsv_v_mean`.

The damage targets are `wire_area_loss_pct` and `ultimate_load_kn`. Candidate models are:

- random forest regressor with `n_estimators=400, min_samples_leaf=2`;
- extra-trees regressor with `n_estimators=500, min_samples_leaf=2`;
- histogram gradient boosting regressor with `learning_rate=0.05, max_depth=5, max_iter=300`;
- MLP regressor with hidden layers (64, 32), ReLU, early stopping, and `max_iter=600`.

The code can optionally add XGBoost or CatBoost if those packages are installed, but the saved metrics files contain only the four scikit-learn candidates listed above. As in the corrosion stage, the final saved models are chosen only by `group_shuffle` MAE. The repository therefore stores random forests for both structural targets.

Two methodological caveats are essential. First, the final state-estimate tables are generated by refitting the selected model on all available structural rows and then predicting all 792 rows. Consequently, structural predictions for the 48 labeled rows are in-sample fits, not held-out validation evidence. Second, the damage stage is not a pure image model; it explicitly uses manual corrosion labels and campaign-linked metadata.

6.3 Degradation Fitting

The degradation stage fits one independent curve per specimen per target for the following four targets:

- observed `surface_total_rust_pct`;
- observed `peak_rust_pct`;
- estimated `estimated_wire_area_loss_pct`;
- estimated `estimated_ultimate_load_kn`.

For each specimen-target pair, the code evaluates linear, exponential, logarithmic, and piecewise-linear forms and chooses the fit with minimum in-sample RMSE. Forecasts are generated from the specimen’s first observed week up to 104 weeks beyond the latest observation in 0.5-week increments. Percentage targets are clipped to $[0, 100]$; load targets are clipped below at 0. No monotonicity, smoothness, or physics constraint is imposed beyond the parametric family itself.

In the saved outputs, piecewise-linear fits dominate: 180 of 192 specimen-target fits use the piecewise-linear family, and the remaining 12 use exponential fits. Neither pure linear nor logarithmic fits are selected in the saved `degradation_curves.parquet`. This selection behavior matters because piecewise-linear models can fit observed histories well while still producing flat or negative terminal slopes.

6.4 Proxy-RUL Estimation

Proxy RUL is computed from the latest row of each specimen plus the saved degradation forecasts. The code first defines a campaign-specific reference load as the maximum observed `ultimate_load_kn` within each campaign. It then computes a health index

$$\text{health} = 1 - (0.30 c + 0.40 w + 0.30 \ell),$$

where

$$c = 0.5 \cdot \text{clip}\left(\frac{\text{surface_total_rust_pct}}{100}, 0, 1\right) + 0.5 \cdot \text{clip}\left(\frac{\text{peak_rust_pct}}{100}, 0, 1\right),$$
$$w = \text{clip}\left(\frac{\text{wire_area_loss_pct}}{30}, 0, 1\right), \quad \ell = \text{clip}\left(1 - \frac{\text{ultimate_load_kn}}{\text{reference_load_kn}}, 0, 1\right).$$

The threshold set is fixed in `configs/default.toml`:

- wire-loss thresholds at 20%, 25%, and 30%;
- load threshold at $0.8 \times$ the campaign reference load;
- health threshold at `health_index` < 0.35.

The saved default proxy RUL is

`estimated_rul_weeks = min(rul_wire_25_weeks, rul_load_threshold_weeks, rul_health_threshold_weeks)`

over the finite, non-missing candidates only. If no threshold is crossed within the forecast horizon, the value remains missing.

The saved `failure_probability` is a proxy score, not a calibrated reliability probability. It is built from a logistic transform of the estimated RUL plus wire-loss, load-margin, and health terms with weights 0.40, 0.25, 0.20, and 0.15 respectively. Risk classes are then assigned by thresholding this score into Low, Moderate, High, and Critical.

7 Split Strategies and Leakage Control

7.1 Why Row-Wise Random Splitting Would Be Invalid

Each specimen contributes a short time series of 15 to 18 images. Random row-wise splitting would place different weeks of the same specimen into both train and test sets. Because the labels evolve gradually within specimen and several metadata fields are constant within specimen, such a split would leak specimen identity and artificially inflate apparent generalization. The repository correctly avoids this by grouping `group_shuffle` on `specimen_id`.

7.2 Implemented Split Regimes

Strategy	Holdout unit	Scientific meaning
<code>group_shuffle</code>	specimen ID	Interpolation to unseen specimens drawn from the same overall mixture of campaigns, treatments, and acquisition conditions. This is the only strategy used for final model selection.
<code>leave_one_treatment_out</code>	treatment code	Extrapolation to an unseen treatment family. In practice this is difficult to interpret cleanly because treatment sizes are highly imbalanced and some treatments exist only in one campaign or only in a few structural rows.
<code>leave_one_campaign_out</code>	campaign ID	Domain-shift test across campaigns, which also changes week horizon, mesh count, NaCl level, and ageing days. This is the sharpest available test of campaign dependence.

Table 10: Split logic implemented in `src/cv/splits.py`.

7.3 Continuous-Target Performance Under Leakage-Aware Splits

Stage	Strategy	Target	Best mean model	MAE	RMSE	R^2
Corrosion	<code>group_shuffle</code>	<code>surface_total_rust_pct</code>	RF	1.381	3.142	0.646
Corrosion	<code>group_shuffle</code>	<code>peak_rust_pct</code>	MLP	4.209	6.234	0.784
Corrosion	<code>leave_one_campaign_out</code>	<code>surface_total_rust_pct</code>	RF	2.096	3.906	0.582
Corrosion	<code>leave_one_campaign_out</code>	<code>peak_rust_pct</code>	HGB	6.211	8.883	0.595
Corrosion	<code>leave_one_treatment_out</code>	<code>surface_total_rust_pct</code>	MLP	1.533	2.659	-0.677
Corrosion	<code>leave_one_treatment_out</code>	<code>peak_rust_pct</code>	RF	4.742	7.245	-0.077
Damage	<code>group_shuffle</code>	<code>wire_area_loss_pct</code>	RF	3.899	4.995	0.310
Damage	<code>group_shuffle</code>	<code>ultimate_load_kn</code>	RF	0.121	0.168	0.385
Damage	<code>leave_one_campaign_out</code>	<code>wire_area_loss_pct</code>	HGB	12.280	15.410	-0.387
Damage	<code>leave_one_campaign_out</code>	<code>ultimate_load_kn</code>	ExtraTrees	0.624	0.655	-8.356
Damage	<code>leave_one_treatment_out</code>	<code>wire_area_loss_pct</code>	HGB	10.759	13.065	-5.481
Damage	<code>leave_one_treatment_out</code>	<code>ultimate_load_kn</code>	RF	0.158	0.189	-4.569

Table 11: Best mean performance by strategy and target, computed from the saved metrics CSV files after averaging over folds within each strategy. RF = random forest, HGB = histogram gradient boosting.

Table 11 supports three distinct conclusions. First, visible corrosion regression generalizes reasonably to unseen specimens within the observed data regime. Second, corrosion regression degrades but remains usable across campaigns. Third, hidden-damage regression is fragile even

under `group_shuffle` and collapses under stricter holdouts, which is consistent with sparse supervision and campaign confounding.

The treatment-holdout results require especially careful interpretation. Some individual treatment folds are easy, but the mean R^2 across treatment holdouts is negative for both corrosion regression targets and strongly negative for both damage targets. A technically defensible summary must therefore use the averaged treatment-holdout result, not the best-looking individual fold.

7.4 Categorical Corrosion Performance

Strategy	Target	Best mean model	Accuracy	Macro-F1
<code>group_shuffle</code>	<code>surface_total_rust_category</code>	Logistic	0.830	0.573
<code>group_shuffle</code>	<code>peak_rust_category</code>	MLP	0.591	0.531
<code>leave_one_campaign_out</code>	<code>surface_total_rust_category</code>	HGB	0.733	0.443
<code>leave_one_campaign_out</code>	<code>peak_rust_category</code>	HGB	0.504	0.441
<code>leave_one_treatment_out</code>	<code>surface_total_rust_category</code>	HGB	0.805	0.464
<code>leave_one_treatment_out</code>	<code>peak_rust_category</code>	HGB	0.598	0.465

Table 12: Best mean classification results from the saved corrosion-classification metrics.

The classification results are less impressive than the surface-category accuracy might suggest. Because the classes are imbalanced, macro-F1 is the safer summary statistic. The repository supports only moderate macro-F1 values, especially once the holdout becomes campaign-level.

8 Results and Interpretation

8.1 Strongly Supported Results

The strongest result is that current visible corrosion can be estimated from image-derived features for unseen specimens under specimen-grouped holdout. The corrosion stage uses only image features, so these results are genuinely image-driven. On `group_shuffle`, the best saved mean metrics are MAE 1.381 and $R^2 = 0.646$ for `surface_total_rust_pct`, and MAE 4.209 with $R^2 = 0.784$ for `peak_rust_pct`. These are credible interpolation results within the observed mixture of campaigns and treatments.

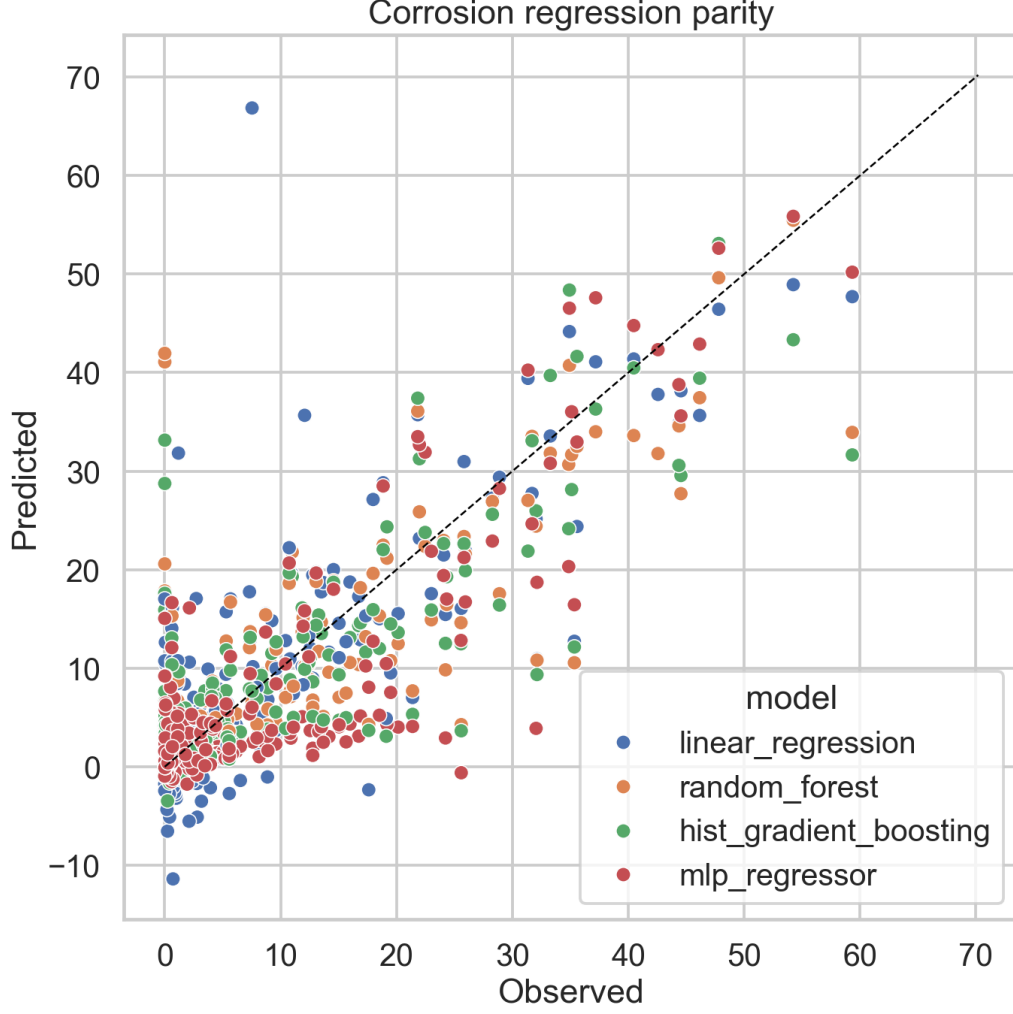


Figure 2: Saved parity plot for corrosion regression under the `group_shuffle` setting. This figure documents the strongest supported visible-corrosion result, not cross-campaign robustness.

The visible-corrosion result does *not* justify claiming that the model has learned internal structural damage, future failure time, or treatment-independent corrosion physics. It supports only current-state visible corrosion prediction under leakage-aware specimen holdout.

8.2 Weak, Negative, and Mixed Results

The damage stage is materially weaker than the corrosion stage. Even on `group_shuffle`, the best saved mean R^2 values are only 0.310 for wire loss and 0.385 for ultimate load. Once the holdout becomes campaign-based or treatment-based, average R^2 becomes negative and, in the case of ultimate load, strongly negative. Those results do not support robust transport of hidden-damage estimation across campaigns or treatments.

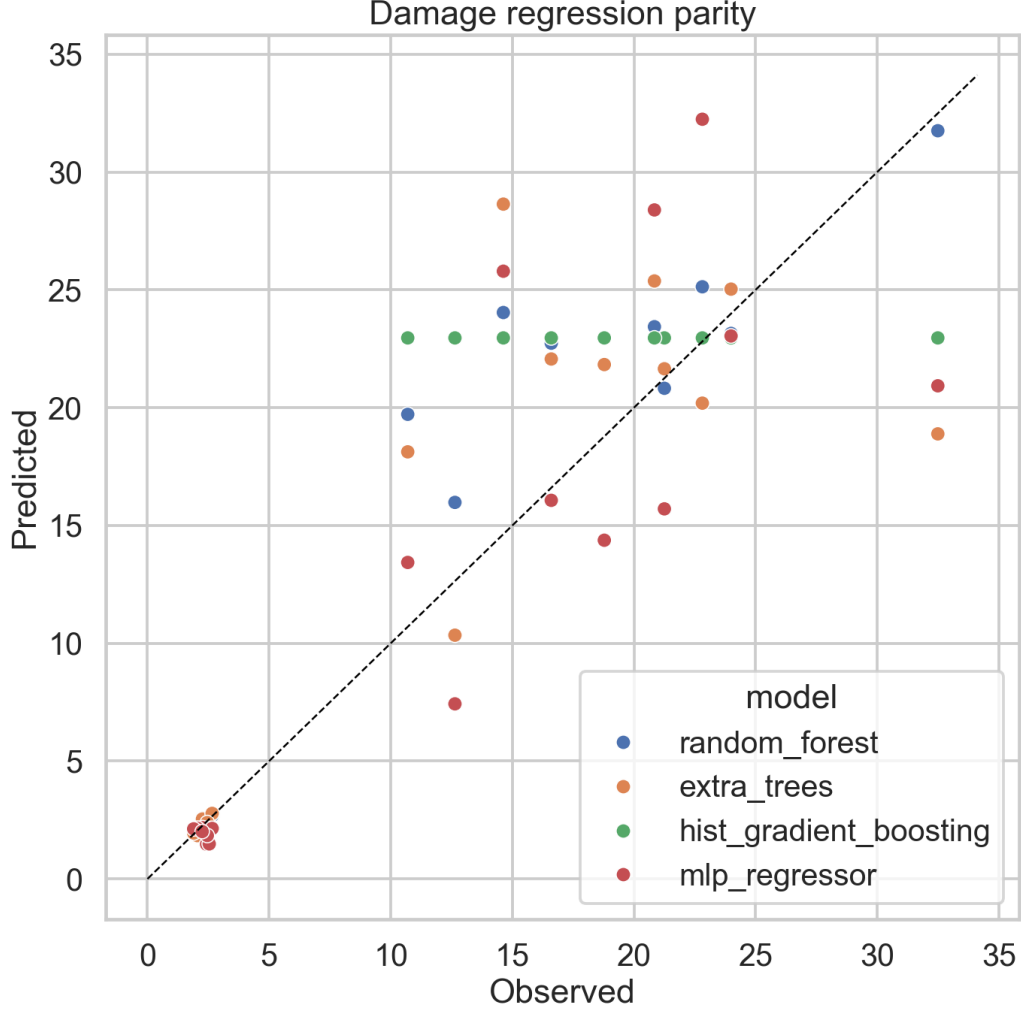


Figure 3: Saved parity plot for damage regression under `group_shuffle`. The plot is useful as an interpolation diagnostic only; it should not be read as cross-campaign or treatment-robust evidence.

The saved full-data damage models also indicate that the two structural targets draw signal from different sources. For wire loss, the largest random-forest importances are `peak_rust_pct_feature` (0.111), `cover_failure_surface_mm` (0.093), `lab_a_mean` (0.081), `rust_component_count` (0.076), and `manual_peak_rust_location_cm` (0.075). For ultimate load, the leading importances are `n_steel_mesh` (0.172), both one-hot campaign indicators (0.128 and 0.105), `week` (0.101), `ageing_days` (0.095), and `nacl_pct` (0.082). This means the current load model is dominated by campaign-linked metadata rather than purely by image evidence.

The structural-label correlations reinforce that point. Across the 48 structural rows, `surface_total_rust_pct` and `wire_area_loss_pct` correlate only 0.021 overall, while `peak_rust_pct` and `wire_area_loss_pct` correlate only 0.011. Meanwhile, `week`, `n_steel_mesh`, `ageing_days`, and `nacl_pct` are perfectly campaign-linked on the structural subset. The repository therefore supports only a weak claim that visible corrosion and selected image descriptors can help estimate hidden damage within the observed dataset; it does not support a strong claim that hidden damage is recoverable from images alone.

8.3 Degradation and Proxy-RUL Outputs

Surface corrosion curves fit observed repeated labels, so their in-sample fit is unsurprisingly good (median R^2 about 0.903 for `surface_total_rust_pct` and 0.916 for `peak_rust_pct`). However, the hidden-damage and load curves are fitted to model-generated estimates rather than repeated measurements. Piecewise-linear models dominate these fits, and the terminal forecast slopes are often flat or negative:

- `estimated_ultimate_load_kn`: 19 positive, 7 zero, 22 negative end slopes;
- `estimated_wire_area_loss_pct`: 10 positive, 32 zero, 6 negative end slopes.

This matters because degradation-to-threshold logic is sensitive to the forecast tail. A flat or negative tail can delay or remove a threshold crossing without any physical justification.

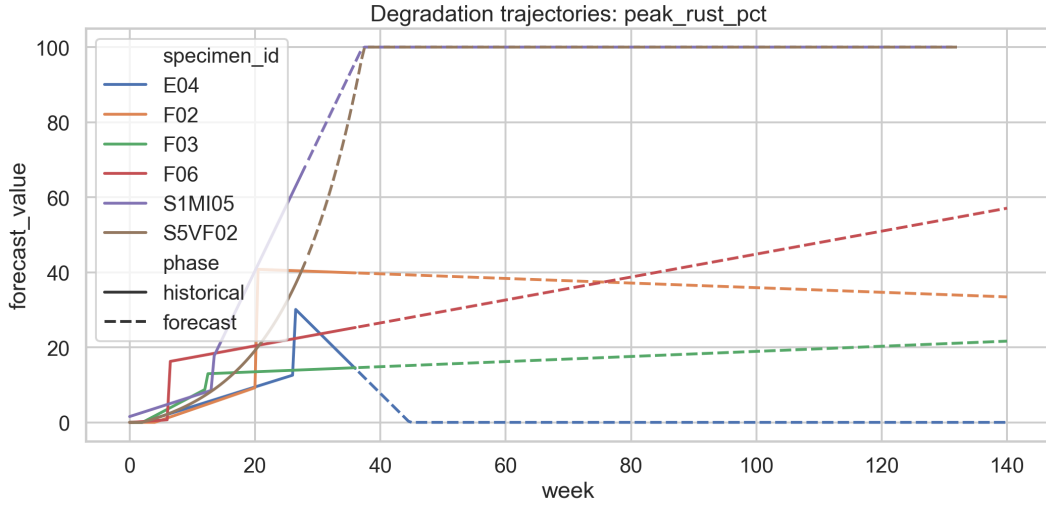


Figure 4: Saved peak-rust trajectory examples for the highest proxy-risk specimens. These are specimen-wise fits and extrapolations, not ground-truth future observations.

Criterion	Finite rows	Zero-week rows	Median weeks	Interpretation
<code>rul_wire_20_weeks</code>	27	25	0.0	Most specimens that ever cross the 20% wire threshold are already at or beyond it in the latest saved state.
<code>rul_wire_25_weeks</code>	20	15	0.0	This is one of the two governing criteria used in the final proxy RUL.
<code>rul_wire_30_weeks</code>	18	7	1.5	Less immediately active than the 25% threshold.
<code>rul_load_threshold_weeks</code>	27	15	0.0	Often governs immediately when load margin is already small.
<code>rul_health_threshold_weeks</code>	10	0	24.0	Rarely active and never the sole earliest governing criterion in the saved outputs.

Table 13: Threshold-crossing behavior in the saved proxy-RUL table.

The specimen-level proxy-RUL outputs are therefore exploratory and conservative in a specific way. Of the 48 specimens, 33 have a finite `estimated_rul_weeks`; among those finite values, 23 are exactly 0.0 weeks and the median is 0.0. Fifteen specimens never cross the chosen thresholds within the 104-week forecast horizon and remain missing. The governing-criterion counts are:

- load only: 16 specimens;
- wire-loss 25% only: 10 specimens;
- both wire-loss and load simultaneously: 7 specimens;
- no finite governing crossing within horizon: 15 specimens.

The health threshold is thus present in code but not materially active in the saved final proxy-RUL decision. The saved risk classes are 20 High, 8 Moderate, 20 Low, and 0 Critical. This is useful as a screening summary, but it does not justify the claim that the repository has learned true remaining life.

9 What the Model Can and Cannot Learn

9.1 What Is Directly Visible

The images directly contain visible rust color, rust spread, local concentration, texture changes, and edge-like deterioration patterns. The corrosion stage is therefore solving a problem that is physically present in the images. The best corrosion models are image-only and do not depend on treatment metadata.

9.2 What Is Only Indirectly Inferable

Wire-area loss and ultimate load are not directly visible in the images. At best, they can be inferred indirectly through correlations with surface corrosion, cover thickness, campaign conditions, and treatment history. The repository confirms this indirectness in two ways:

- structural supervision exists only once per specimen and only at the end of observation;
- the saved hidden-damage models use metadata and manual corrosion labels, not just image-derived evidence.

This means the current damage stage is estimating latent structural state from a mixed evidence bundle, not discovering a purely image-based structural law.

9.3 What Is Hidden and Not Reliably Recoverable Here

True failure time is hidden and unrecoverable from the current dataset because it is never observed. More subtly, structural progression between week 0 and the final destructive test is also hidden: the dataset does not contain repeated direct measurements of wire loss or load capacity over time. The repository therefore fills that gap with model-generated hidden-damage estimates and then smooths them with curve fitting. That is a defensible exploratory engineering approximation, but it is not the same as observing hidden damage.

9.4 Why Some Targets Are Easier Than Others

Visible corrosion targets are easier because they are dense (792 labels), directly image-linked, and observed at every time point. Hidden-damage targets are harder because they are sparse (48 labels), terminal only, partially confounded with campaign, and physically not fully visible on the surface.

9.5 Why Metadata Can Outperform Image Features

In the structural subset, metadata fields encode campaign identity almost perfectly. For ultimate load, the saved full random forest draws most of its importance from mesh count, campaign indicators, week, ageing days, and NaCl level. In other words, the current model can partially exploit experimental design structure. Scientifically, that means metadata can look strong even when image-driven transportability is weak.

9.6 Scientific Implication

The most defensible scientific interpretation of `main_3` is not “images predict structural life.” The stronger statement is: the repository demonstrates that visible corrosion state can be learned from images, and it assembles an exploratory condition-assessment stack that maps visible corrosion plus metadata to hidden-damage proxies and threshold-status indicators. That is useful, but it is not the same as validated structural prognosis.

10 Traceability to the Original Thesis and Documentation

The thesis chapter on automated image analysis and the conference paper both describe a method family that is related to `main_3` but not identical to it. The thesis aligns with the current repository on the total number of images (792) and on the broad goal of color-based corrosion quantification. The conference paper reports 760 prepared images and a three-class classifier, so it corresponds to an earlier or narrower snapshot than the current repository.

Table 14: Traceability from documented thesis/conference methods to the current repository implementation.

Documented method element	What the documentation says	What the repository actually does	Expected consequence of mismatch
Preprocessing toolchain	Thesis chapter 7 describes manual and batch GIMP/BIMP preprocessing with resize to 3555×886 px, crop to 2835×650 px, and white-balance level adjustment using input range [0, 195] and output [0, 255]	<code>main_3</code> loads the existing PNG files, crops by relative ratios, rescales channel intensities by percentiles, and uses adaptive histogram equalization in Python	Color and geometry normalization are not thesis-equivalent. Any mismatch in feature-target alignment can partly arise from preprocessing differences.

Continued on next page

Documented method element	What the documentation says	What the repository actually does	Expected consequence of mismatch
Segmentation logic	Thesis and conference paper describe MATLAB RGB thresholding with rust mask [25, 0, 0] to [255, 100, 80] plus explicit black-pore and gray-mark exclusion masks	<code>main_3</code> uses a weighted RGB/HSV/Lab rust-likelihood score, adaptive thresholding, and morphology cleanup	The current mask is more adaptive but less directly traceable to the documented thresholds. Quantitative agreement with thesis overlays cannot be assumed.
Spatial strip analysis	Thesis describes 1 cm strip analysis with overlap, averaging about 47 strips over a 24 cm specimen; conference paper mentions 0.5 cm strips	<code>main_3</code> uses 12 non-overlapping longitudinal segments over 24 cm	Current spatial features are coarser and structurally different from both documented strip schemes. Direct equivalence of peak-location or peak-rust calculations is unsupported.
Target formulation	Conference paper trains a three-class low/medium/high corrosion classifier; thesis discussion centers on segmented rust area and strip peaks	<code>main_3</code> trains regressors for spreadsheet corrosion percentages and classifiers for spreadsheet 1–5 category labels	The repository is solving a different supervised task from the documented three-class classifier, even though the physical topic is related.
Learning model	Conference paper describes transfer learning on ResNet-50	<code>main_3</code> uses handcrafted feature extraction plus classical ML; the older deep baseline is in sibling project <code>main_2</code> , not in the saved <code>main_3</code> pipeline	Reported <code>main_3</code> results should not be presented as direct reproduction of the documented ResNet workflow.
Automation artifacts	Thesis states that full MATLAB code and GIMP/BIMP configuration exist as attachments	Those exact MATLAB and BIMP source files are not present in <code>main_3</code> ; only the current Python reimplementations are available	Exact thesis preprocessing and mask-generation reproducibility cannot be verified from the repository alone.

Continued on next page

Documented method element	What the documentation says	What the repository actually does	Expected consequence of mismatch
Dataset provenance	Conference paper states 760 prepared images; thesis states 792 preprocessed images	Current repository contains 792 rows and 792 PNGs, matching the thesis count	The current repository is closer to the thesis dataset snapshot than to the conference snapshot, but the snapshots are not perfectly interchangeable.

The traceability conclusion is therefore mixed. The current repository is clearly thesis-inspired in its use of corrosion-focused image analysis and strip-like localization logic, but it is not an exact code reproduction of the thesis GIMP+MATLAB workflow. The current implementation should be described as a Python reconstruction and extension, not as exact reproduction.

11 Limitations

The limitations are not generic; they are tied directly to the present repository state.

- **Small structural supervision.** Only 48 rows carry structural labels, and each specimen contributes exactly one structural row. This is the dominant bottleneck.
- **Campaign dependence.** The two campaigns differ simultaneously in week horizon, mesh count, NaCl level, ageing days, and treatment composition. Several metadata fields become campaign surrogates.
- **Treatment imbalance.** Structural rows per treatment are highly uneven, from 2 rows for VF and 2 for SA_VF up to 18 for SA. Leave-one-treatment-out damage regression is therefore very unstable.
- **No direct RUL supervision.** The dataset contains no failure times, no censoring labels, and no experimentally observed end-of-life events, so only threshold-based proxy RUL is possible.
- **Downstream manual-label dependence.** Hidden-damage estimation is not end-to-end image-only because manual corrosion labels are used as damage-model inputs.
- **Model-on-model trajectories.** Hidden-damage and load degradation curves are fitted to model estimates at all weeks rather than to repeated structural measurements.
- **No monotonicity constraint.** The curve fitting can produce flat or negative progression rates, especially for hidden-damage proxies and load.
- **Preprocessing uncertainty.** The thesis workflow described in documentation is not exactly reproduced. This likely contributes to the weak direct correlation between intuitive mask-based proxies and the manual corrosion labels.
- **Category-definition ambiguity.** The repository stores ordinal corrosion categories but does not define the quantitative cutoffs behind the 1–5 scale.
- **Single corrupted image retained.** One unreadable PNG remains in the dataset with a failure flag and median-filled features. This is minor globally but real.

- **External validity.** The saved models are calibrated only on 48 ferrocement specimens across two campaigns. Generalization beyond that regime is untested.

12 Future Work

Only follow-up work that directly addresses the observed weaknesses is justified.

- Recover or reimplement the exact thesis preprocessing attachments (GIMP/BIMP settings and MATLAB masks) and quantify feature-level agreement against the current Python pipeline.
- Separate the downstream damage stage into explicit ablations: metadata-only, manual-corrosion-only, image-only, and mixed-input models. The current repository cannot isolate these contributions.
- Generate out-of-fold hidden-damage estimates for all rows before fitting degradation curves, so that downstream trajectory fitting does not rely on in-sample refits over the same structural labels.
- Collect repeated structural measurements before the destructive endpoint. Even a small number of intermediate wire-loss or load observations would materially improve damage progression modeling.
- Add monotone or physics-informed trajectory constraints for wire loss and residual capacity, because the current best-RMSE fits frequently flatten or reverse at the forecast tail.
- Define and document the 1–5 corrosion-category rules in the repository, or replace them with transparent threshold labels derived from explicit numeric rules.
- Build a specimen-treatment mapping table that decouples series naming from treatment identity, so future analyses cannot accidentally infer treatment from filenames.
- Validate the threshold-based RUL outputs against experimentally observed threshold crossings in future campaigns, rather than only against model-generated forecasts.

A Meeting-Defense Questions and Concise Evidence-Based Answers

Why not direct RUL? Because neither the spreadsheet nor the outputs contain true failure times. The repository only has current corrosion labels at every image and structural labels at one late-stage row per specimen. The saved `estimated_rul_weeks` value is therefore a threshold-crossing time computed from forecasted states, not a supervised failure-time target.

Why these split strategies? Random row-wise splitting would leak different weeks of the same specimen across train and test. `group_shuffle` prevents that by holding out whole specimens. Leave-one-treatment-out and leave-one-campaign-out are stricter stress tests that expose domain dependence and shortcut learning.

Why did robustness drop under stricter holdouts? Because the data-generating conditions change with treatment and especially with campaign. In the structural subset, week, mesh count, ageing days, and NaCl level are effectively campaign markers. Once those markers are held out, performance collapses, especially for hidden-damage targets.

Why can visible corrosion be predicted better than structural degradation? Visible corrosion is directly present in the images and has 792 supervised rows. Structural degradation is only sparsely observed, is mostly hidden beneath the surface, and has only 48 late-stage labels.

How much of the downstream signal comes from metadata? The repository does not contain a clean ablation study, so the exact fraction cannot be recovered from saved outputs alone. However, the saved full ultimate-load random forest is dominated by metadata and campaign indicators, which is strong evidence that metadata contributes materially.

Are the corrosion results partly tautological? Partly, in the sense that the features were engineered to respond to rust-colored regions. But the current Python features are not numeric copies of the manual labels: `rust_area_pct_feature` correlates only 0.257 with `surface_total_rust_pct`, and `peak_rust_pct_feature` correlates only 0.183 with `peak_rust_pct`. So the model is not simply reading back an identical mask.

Could preprocessing mismatch explain part of the failure? Yes. The thesis documents a GIMP/BIMP plus MATLAB RGB-threshold workflow, while `main_3` uses a Python adaptive-color pipeline. That mismatch is a plausible explanation for why intuitive area/peak proxies align only weakly with the manual corrosion labels.

What are the highest-risk assumptions in the repository? The highest-risk assumptions are: scaling raw wire-loss values by 100, treating hidden-damage trajectories as smooth fits to model-generated estimates, and using campaign-linked metadata inside structural proxy models without a clean causal disentangling step.

Which features are currently the most interpretable? The most interpretable families are the rust-mask area, segment-wise rust distribution, peak-rust location, connected-component geometry, and color statistics. They can be tied to visible surface patterns. Their weakness is that they are sensitive to segmentation quality and preprocessing consistency.

What exactly do the important metrics mean? MAE is mean absolute error in target units. RMSE penalizes larger errors more strongly. R^2 measures explained variance and can become negative when the model is worse than predicting the mean. Macro-F1 averages per-class F1 scores and is more reliable than accuracy under class imbalance. `progression_rate_per_week` is the final forecast slope of a fitted curve. `health_index` is a weighted score from corrosion, wire loss, and load margin. `failure_probability` is only a proxy risk score. `estimated_rul_weeks` is weeks to the earliest selected threshold crossing, not weeks to true failure.

What is the shortest defensible description of the whole system? `main_3` is a repository-grounded corrosion-to-threshold-status pipeline. It gives credible image-based visible corrosion estimates for unseen specimens under related conditions, but hidden-damage estimation and proxy RUL remain exploratory because the structural supervision is sparse, campaign-linked, and not true failure-time data.